VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



CO5085 EXERCISES REPORT

DEEP LEARNING AND ITS APPLICATIONS IN COMPUTER VISION

Group: Traiblazer

Instructors: Dr. Le Thanh Sach
Dr. Nguyen Quoc Minh

Student: Dinh Hoang Duy Khanh 2670306

Ho Chi Minh City, November 2026

Member List and Workload

No.	Name	Student ID	Main responsibilities	Contribution
1	Dinh Hoang Duy Khanh	2670306	Build and run E1–E3, create figures, verify results, prepare the GitHub page, and write this report.	100%

Contents

Member List and Workload **2**

Introduction **4**

1 E1 – Softmax, MLP, and CNN **5**
1.1 Objective and model design 5
1.2 Results 5
1.3 Error analysis 6

2 E2 – Multi-Head Self-Attention for Images **8**
2.1 Tokens and attention 8

3 E3 – LSTM and GRU for Image Sequences **11**
3.1 Image as a sequence 11
3.2 Interpretation 12

4 Requirement Checklist and Conclusion **13**

Introduction

This report completes Exercises E1, E2, and E3 of CO5085 with one shared small dataset: Fashion-MNIST. Fashion-MNIST contains 28×28 grayscale clothing images in ten classes. It is allowed by the exercise sheet, visually more varied than handwritten digits, and small enough for a beginner to run on a personal computer.

The official 60,000 training images are shuffled with seed 42, then split into 48,000 training images and 12,000 validation images. The official 10,000 test images are kept for final evaluation. The same split is used in every exercise, so comparisons are fair. All models use Adam with learning rate 0.001, batch size 256, and cross-entropy loss. E1 and E3 use three epochs. E2 uses two epochs because it contains three attention experiments. These short runs compare designs clearly; they do not claim a fully optimized benchmark score.

How to read the results. Validation accuracy is used during training to compare choices. Test accuracy is reported after an experiment completes. A parameter count is the number of learned values in a model. A confusion matrix shows which classes are confused instead of only showing one average score.

Reproducibility and AI-use declaration

The project uses direct PyTorch code. Run `python -m E1.run_e1`, `python -m E2.run_e2`, and `python -m E3.run_e3` from the repository root. The first run downloads Fashion-MNIST. Figures and JSON metrics are saved in `reports/`. AI assistance was used to help organize code and explain concepts. The student reviewed the code, ran the experiments, and is responsible for understanding and defending every model, result, and conclusion in this report.

1 E1 – Softmax, MLP, and CNN

1.1 Objective and model design

E1 requires three classifiers and a custom training loop. The shared loop in `common.py` is explicit: for each batch it predicts scores, computes cross-entropy loss, calls `loss.backward()`, then calls `optimizer.step()`. This makes the learning process visible instead of hiding it inside a high-level trainer.

The Softmax classifier flattens an image into 784 pixel values and uses one linear layer with ten outputs. The MLP also flattens the image, then uses hidden layers of 128 and 64 values with ReLU activation. The CNN keeps the image grid. It applies convolution, ReLU, and max-pooling twice before a small fully connected head. Convolution helps the CNN learn local edges and clothing shapes from nearby pixels.

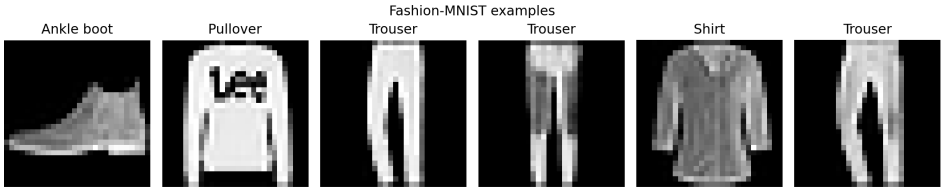


Figure 1.1: Examples from Fashion-MNIST. Each image has one clothing label.

1.2 Results

Table 1.1: E1 comparison on the same test set. Time includes training and evaluation.

Model	Parameters	Time (s)	Test accuracy
Softmax linear	7,850	125.2	0.8258
MLP	109,386	103.2	0.8518
CNN	105,866	156.4	**0.8686**

The CNN is best at 0.8686. It has nearly the same number of parameters as the MLP, so the improvement is not simply from a larger model. The CNN keeps local two-dimensional relationships; the MLP loses them when it flattens the image.

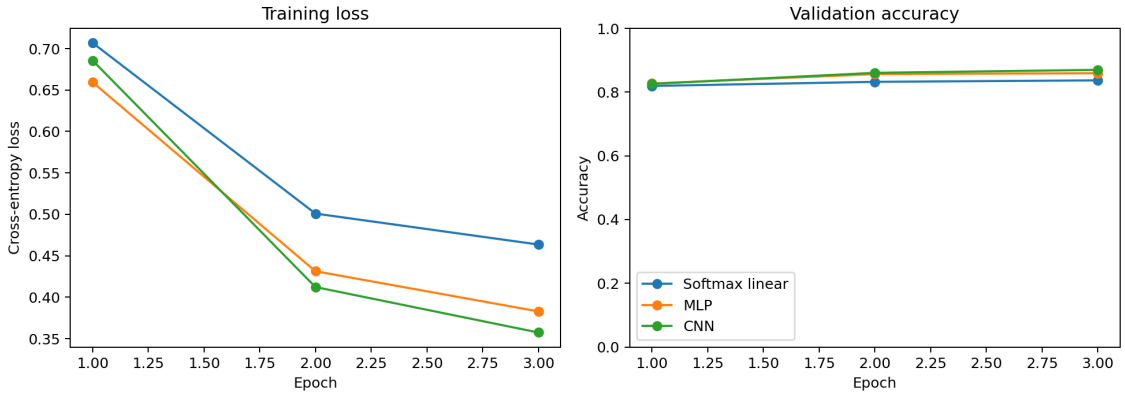


Figure 1.2: Training loss and validation accuracy for E1.

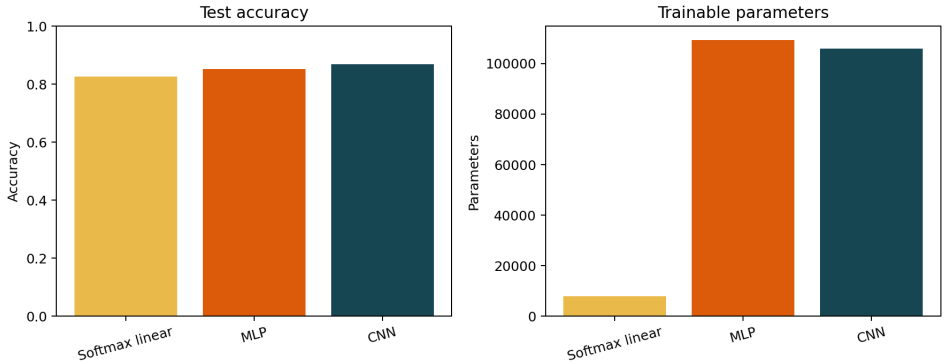


Figure 1.3: E1 test accuracy and parameter comparison.

1.3 Error analysis

The CNN confusion matrix in Figure 1.4 is calculated from the final test predictions. The diagonal contains correct predictions. Most off-diagonal errors are between visually similar upper-body classes such as shirt, T-shirt/top, pullover, and coat. This is reasonable because the images are grayscale and low resolution. Figure 1.5 shows individual errors, which explains the score better than accuracy alone.

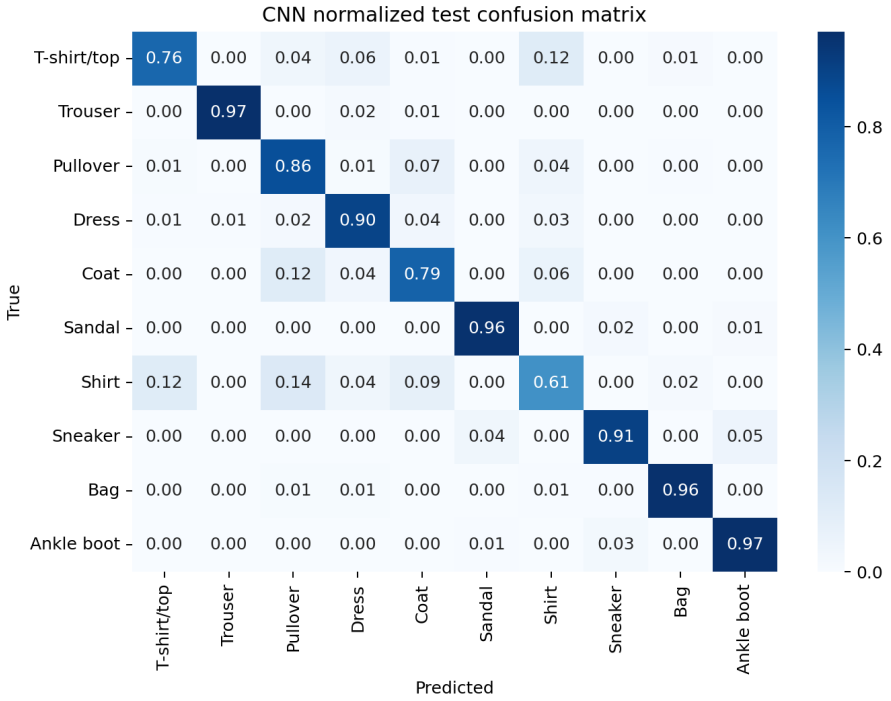


Figure 1.4: Confusion matrix for the selected E1 CNN.

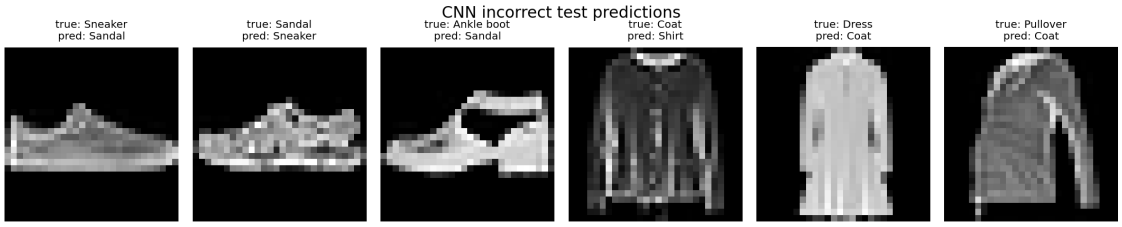


Figure 1.5: A small selection of CNN errors. Titles give true and predicted labels.

2 E2 – Multi-Head Self-Attention for Images

2.1 Tokens and attention

Attention first needs a sequence of image tokens. The patch tokenizer cuts an image into 49 non-overlapping 4×4 patches and projects each patch to 32 values. The CNN-stem tokenizer uses two small convolution-and-pooling stages, then converts its 7×7 feature map into 49 tokens. The sequence length is the same, but the CNN stem supplies tokens that already contain local visual features.

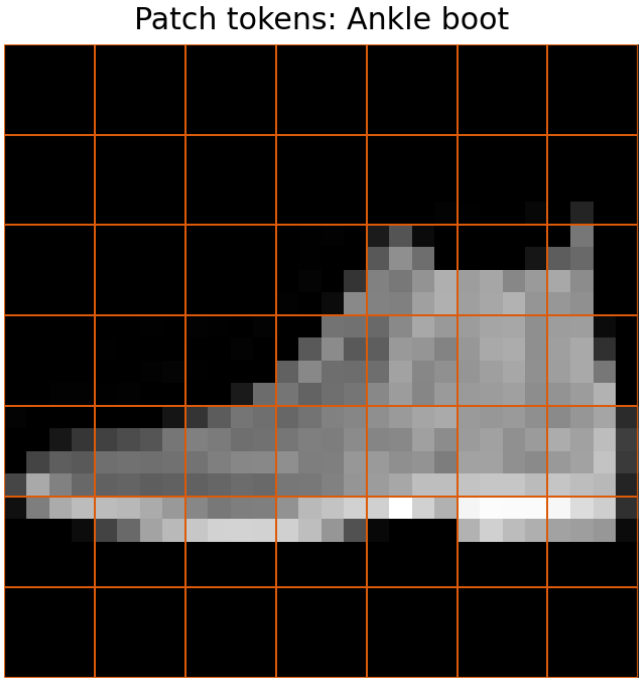


Figure 2.1: One image divided into 49 patch tokens.

The manual MSA uses only linear layers, matrix multiplication, transpose, softmax, and reshape. For tokens X, it forms $Q = XW_Q$, $K = XW_K$, and $V = XW_V$. Attention weights are $\text{softmax}(QK^T/\sqrt{d})$ and the output is a weighted sum of V. The code splits 32 values into two heads, computes each head, and joins them. It does not call `nn.MultiheadAttention`. The next experiment uses PyTorch MSA with the same patch tokens and settings. The third experiment changes only the tokenizer to the CNN stem.

Table 2.1: E2 attention and tokenization comparison; all runs use two epochs.

Model	Parameters	Time (s)	Test accuracy
Manual MSA + patch	10,986	110.7	0.7061
PyTorch MSA + patch	10,986	95.5	0.7072
PyTorch MSA + CNN stem	15,242	119.1	**0.7578**

Manual and PyTorch MSA with patch tokens have almost the same accuracy, 0.7061 and 0.7072. This is expected because their tokenization and architecture are matched. PyTorch MSA is a little faster. The CNN-stem tokenizer improves test accuracy to 0.7578 because it adds local edge and texture features before attention. The attention models remain below the E1 CNN because they are shallow and trained for only two epochs without augmentation or pretraining.

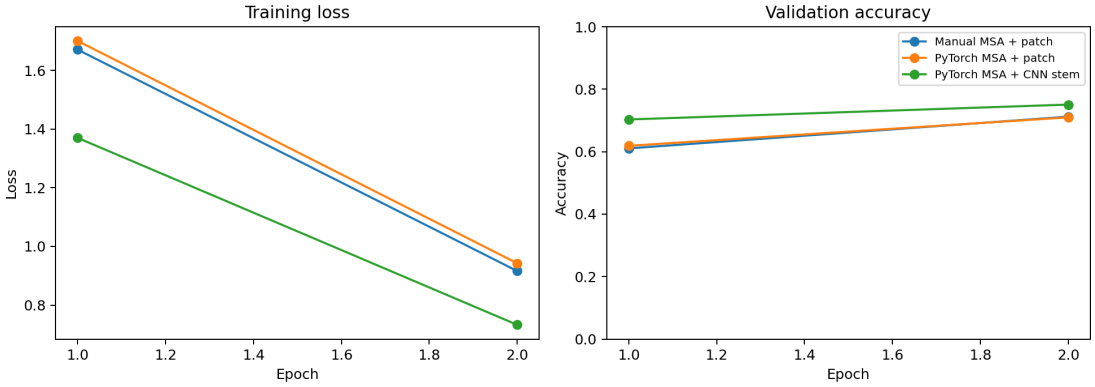


Figure 2.2: E2 learning curves.

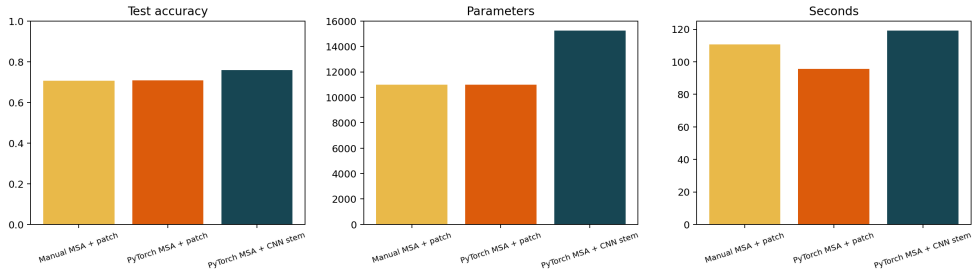


Figure 2.3: E2 comparison of accuracy, parameters, and time.

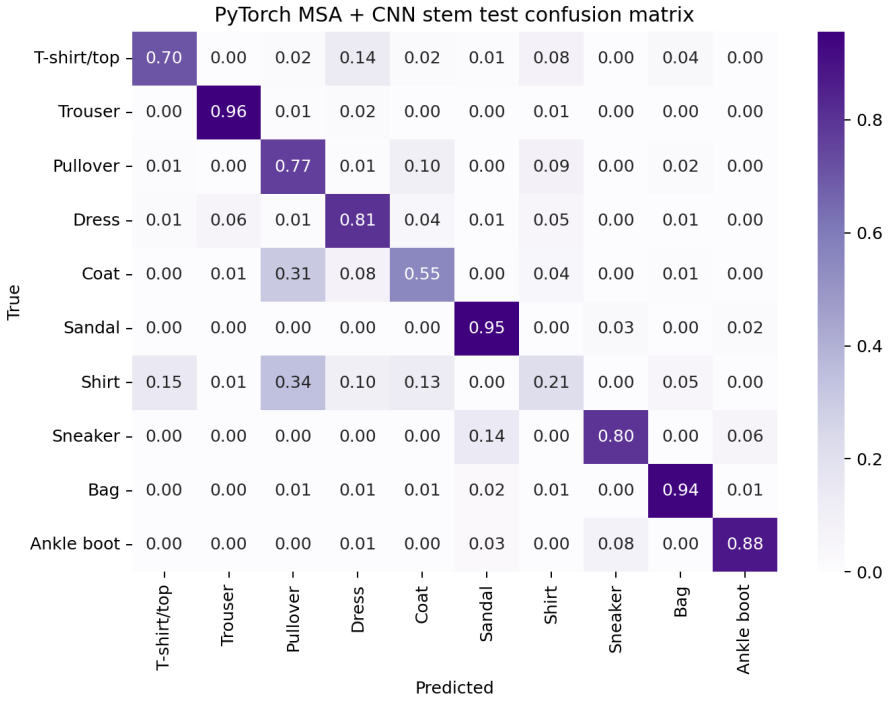


Figure 2.4: Confusion matrix for PyTorch MSA with CNN-stem tokens.

3 E3 – LSTM and GRU for Image Sequences

3.1 Image as a sequence

E3 reads an image as a sequence instead of a grid. Each of the 28 image rows is one time step, and every step contains 28 normalized pixels. The sequence therefore has length 28 and input size 28. This is simple to implement, but it has a limitation: a recurrent model must compress earlier rows into a hidden state and does not naturally inspect nearby pixels in both directions as a CNN does.

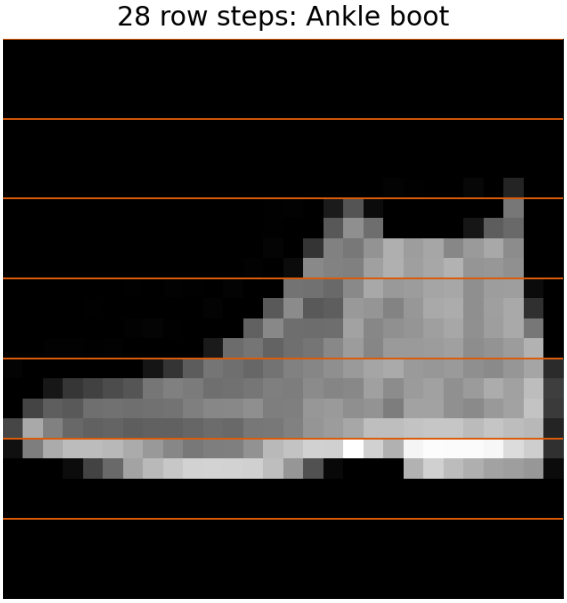


Figure 3.1: Rows marked as time steps for LSTM and GRU.

LSTM and GRU both use hidden size 64 and a final linear classifier. The last recurrent output predicts the clothing class. LSTM has separate gates and a cell state, so it has more parameters. GRU combines some gates, so it is smaller. The E1 CNN result is the image-grid baseline.

Table 3.1: E3 recurrent models and the CNN baseline.

Model	**Parameters**	**Time (s)**	**Test accuracy**
LSTM rows	24,714	131.1	0.8201
GRU rows	18,698	102.6	0.8156
CNN baseline (E1)	105,866	156.4	**0.8686**

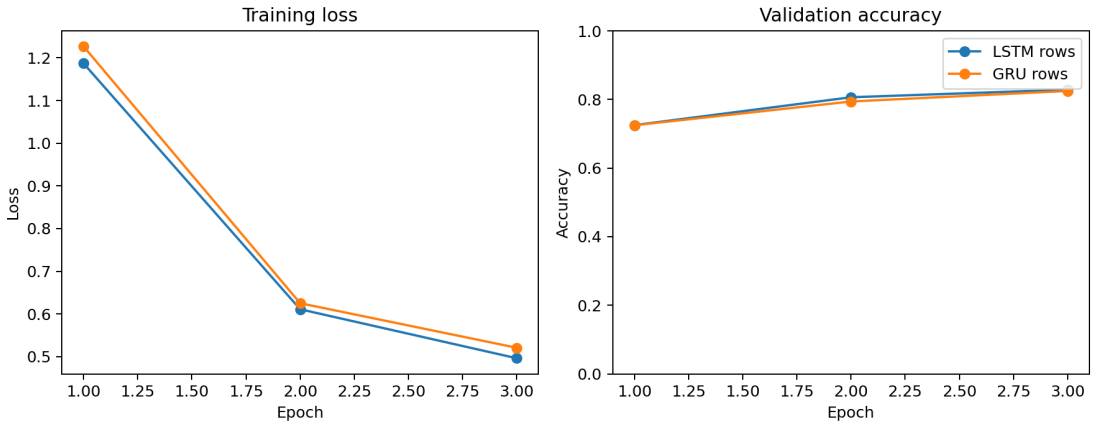


Figure 3.2: LSTM and GRU training loss and validation accuracy.

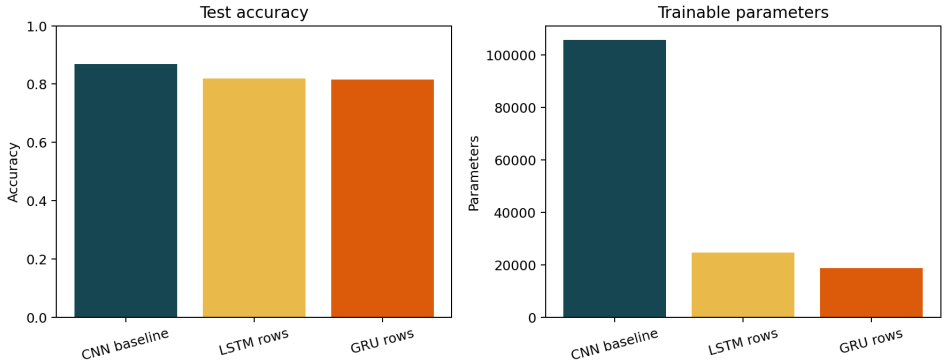


Figure 3.3: E3 accuracy, parameter, and time comparison.

3.2 Interpretation

LSTM gives 0.8201 test accuracy and GRU gives 0.8156. GRU is faster and has fewer parameters, while LSTM is slightly more accurate. Both are below the CNN at 0.8686. The result supports the representation argument: CNN sees nearby pixels in height and width, while the row-sequence model has to remember earlier rows in a hidden state. RNNs are natural for sequence data; for these small images, CNN is the better choice.

4 Requirement Checklist and Conclusion

Table 4.1: Evidence that the exercise requirements are completed.

Requirement	Evidence
One shared dataset	Fashion-MNIST and the same 48,000/12,000/10,000 split are used in E1–E3.
E1 models and errors	`E1/run_e1.py`, Table 1.1, curves, confusion matrix, and error samples.
Manual training loop	`common.py` explicitly runs forward, loss, backward, and optimizer step.
Manual and library MSA	`ManualMSA` uses matrix operations; the library comparison uses `nn.MultiheadAttention`.
Two tokenizations	Patch tokens and CNN-stem feature-map tokens; Table 2.1.
LSTM, GRU, baseline	`E3/run_e3.py` compares both recurrent units with the E1 CNN.
Reproducible delivery	Code, figures, JSON results, README, GitHub Pages source, and this PDF are stored together.

This project shows three ways to classify the same images. Softmax is the simplest baseline, the MLP learns nonlinear pixel combinations, and the CNN is strongest because it keeps the image layout. Manual attention agrees closely with PyTorch MSA, and CNN-stem tokens improve over raw patches. LSTM and GRU show how to read an image as a sequence, but CNN remains more suitable for this image dataset.

The code is intentionally short and direct. It shows the essential steps: prepare data, define a model, train with forward and backward propagation, evaluate a held-out test set, and inspect errors. Future work could add more epochs, augmentation, a deeper CNN, or a larger attention model. Those changes are outside this beginner-level submission.